Software Requirements Specification (SRS)

1. 📌 Introduction

1.1 Purpose of the System

The purpose of the Price and Quality Comparison (PQC) application is to provide a unified platform that automatically aggregates, compares, and analyzes e-commerce products. This system allows consumers to make informed purchasing decisions by finding the most cost-effective deals while simultaneously evaluating product quality through AI-driven sentiment analysis. This document serves as the architectural blueprint for the Final Year Project (FYP), specifically emphasizing a MERN stack implementation.

1.2 Scope

The system will target the Pakistani e-commerce market, focusing on major retailers like Daraz, PriceOye, Shophive, Naheed, and Telemart. The scope encompasses distributed web scraping utilizing Puppeteer, product matching across disparate catalogs, data storage in MongoDB, Natural Language Processing (NLP) for bilingual (Urdu and English) search processing and sentiment analysis, and a ReactJS front-end.

1.3 Definitions / Acronyms

- **PQC:** Price and Quality Comparison.
- **MERN:** MongoDB, Express.js, ReactJS, Node.js framework stack.
- **NLP:** Natural Language Processing.
- **SPA:** Single Page Application.
- **DOM:** Document Object Model.

1.4 References

- Puppeteer Web Scraping Guidelines for Node.js.
- Kaggle E-Commerce Datasets API Integration.
- Roman Urdu to Urdu Transliteration and NLP Sentiment Libraries.

2. 🎯 System Overview

2.1 System Description

The PQC system follows a modern client-server architecture built entirely on JavaScript technologies. A ReactJS frontend allows users to search for products in either English or Urdu. An Express.js/Node.js backend processes these requests, utilizing NLP to translate and transliterate queries. The backend dispatches scraping jobs via Puppeteer workers to extract dynamic content from e-commerce SPAs. Extracted data is stored in a highly scalable MongoDB database.

2.2 Objectives

- To centralize market prices across various Pakistani online stores.
- To enable seamless bilingual product searching (English and Urdu) using NLP query translation.
- To provide an objective "Quality Score" by classifying English and Roman Urdu user reviews into positive or negative sentiments.
- To implement a robust Node.js backend capable of scraping JavaScript-rendered retail websites safely.

2.3 Target Users

- **Consumers:** Everyday shoppers looking for the best electronics, groceries, and fashion.
- **Bargain Hunters:** Users monitoring flash sales and historical price drops.
- **System Admin:** Managers monitoring scraping health, proxy rotations, and database scaling.

3. 👥 Users

- **Guest User:** Can search for products in Urdu/English, view comparisons, and browse the Product Details Page.
- **Registered User:** Can save items to a wishlist, leave platform reviews, and set price alerts.
- **Admin:** Can manually trigger Puppeteer scraping jobs, update target stores, and seed datasets from Kaggle.

4. ⚙️ Functional Requirements

4.1 Product Search (Urdu & English NLP)

The system shall allow users to search for products using keywords in English, Urdu script, or Roman Urdu (e.g., "Mujhe Samsung mobile chahiye"). The backend will utilize an NLP transliteration library to convert Roman Urdu to standard terms (e.g., converting to "مجھے سام سنگ موبائل چاہیے" or translating to English) to effectively query the database and trigger targeted web scraper searches.

4.2 Price Comparison

The system shall display side-by-side pricing for identical products across different platforms. The backend will use a Product Matching Engine that cross-references extracted titles, attributes, and brand names to link the same product sold on Daraz with its counterpart on PriceOye.

4.3 Quality Comparison (Ratings/Reviews)

The system shall scrape user reviews from target platforms. It will employ NLP libraries designed for Multilingual and Roman Urdu text (using models like TF-IDF combined with SVM classifiers) to evaluate sentiment and calculate an aggregate "Quality Score".

4.4 Filters & Sorting

The UI shall allow users to sort results by lowest price, highest quality score, brand, and target store.

4.5 Wishlist / Favorites

Registered users shall be able to save products to their account, allowing the database to track price changes over time for those specific items.

4.6 Product Details Page (PDP)

Clicking a search result will open a comprehensive Product Details Page. This page will display high-resolution images, a unified list of specifications (RAM, color, etc.), a graphical timeline of price history, and the aggregated NLP sentiment breakdown of user reviews.

4.7 Admin Management Features & Dataset Seeding

The admin dashboard shall provide tools to manually seed the database using Kaggle e-commerce and review datasets. This allows the NLP models and front-end interface to be trained and tested without waiting for live scraping.

5. 🚫 Non-Functional Requirements

5.1 Performance

Database reads for existing products must execute in under 300ms. Live Puppeteer scrapes must run asynchronously to prevent blocking the Node.js event loop.

5.2 Scalability

The MongoDB database must dynamically accommodate diverse product structures without schema migrations. The system should support horizontal scaling of Node.js scraping workers.

5.3 Security

- **Anti-Bot Evasion:** Puppeteer workers must use stealth plugins and intercept network requests (blocking unnecessary fonts and images) to reduce memory footprint and avoid triggering Web Application Firewalls (WAFs).
- **Authentication:** User sessions must be secured using JWT (JSON Web Tokens).

5.4 Usability

The ReactJS frontend must provide a responsive, mobile-first design, specifically optimized for users reading dual-language (Urdu/English) inputs.

5.5 Availability

The system must be highly available, utilizing a message queue buffer so that if a Puppeteer worker crashes during a scrape, the URL task is safely preserved and retried.

6. 🧱 System Architecture

6.1 High-Level Architecture (Frontend + Backend + DB)

1. **Frontend (ReactJS):** Handles the UI, state management, and renders data charts.

2. **Backend (Node.js & Express.js):** Acts as the API gateway, handles user authentication, and runs the NLP transliteration pipelines.
3. **Queue/Broker:** Redis manages the backlog of URLs awaiting scraping.
4. **Extraction Workers:** Distributed Node.js processes running Puppeteer to navigate target sites, execute search engine queries, and parse the DOM.
5. **Database:** MongoDB clusters storing the scraped, semi-structured JSON product data.

6.2 Technology Stack

- **Frontend:** ReactJS, TailwindCSS, Chart.js.
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB, Mongoose (ODM).
- **Scraping:** Puppeteer (Headless Chrome), Cheerio (for fast static HTML parsing where JS isn't needed).
- **NLP/AI:** Python microservice or Node.js NLP libraries for Roman Urdu-to-English transliteration and sentiment classification.

6.3 Third-Party Integrations

- **Kaggle API:** Used for initially bootstrapping the MongoDB database with bulk e-commerce items to test the UI and NLP models.
- **Proxy Networks:** Integrated into Puppeteer to route requests through residential IPs, bypassing rate limits on major Pakistani stores.

7. 🗂️ Data Models (MongoDB Schema Design)

Unlike strict relational databases, MongoDB's document model allows us to store the highly variable attributes of different e-commerce items seamlessly in BSON format.

7.1 Product

The core item document containing embedded details for fast reads.

- `_id` (ObjectId)
- `title` (String)
- `brand` (String)
- `category` (String)
- `specifications` (Object) - *Dynamic key-value pairs (e.g., { "RAM": "8GB", "Material": "Cotton" })*
- `price_sources` (Array of Objects) - *Embedded array linking the identical item across Daraz, Naheed, etc., for rapid loading on the Product Details Page.*

7.2 Price Source (Embedded in Product or Referenced)

- `store_name` (String: e.g., 'Daraz', 'PriceOye')
- `url` (String)
- `current_price` (Number)
- `historical_prices` (Array of Objects: `{ price: Number, date: Date }`)

7.3 Review

Stored in a separate collection to prevent the Product document from exceeding MongoDB size limits if there are thousands of reviews.

- `_id` (ObjectId)
- `product_id` (ObjectId -> Refers to Product)
- `original_text` (String - Urdu/English)
- `transliterated_text` (String)
- `sentiment_score` (Number: e.g., -1.0 to 1.0)
- `sentiment_label` (Enum: Positive, Neutral, Negative)

7.4 User

- `_id` (ObjectId)
- `email` (String)
- `password_hash` (String)
- `wishlist` (Array of Product ObjectIds)

8. 🔌 API Design

8.1 Authentication APIs

- `POST /api/v1/auth/register` - Registers a user.
- `POST /api/v1/auth/login` - Authenticates and returns a JWT.

8.2 Product APIs

- `GET /api/v1/products/search?q={query}&lang={ur|en}` - Triggers the NLP engine to translate the query, searches MongoDB, and dispatches a Puppeteer job if the data is stale.
- `GET /api/v1/products/{id}` - Fetches all data required to render the Product Details Page.

8.3 Price APIs

- `POST /api/v1/products/{id}/alerts` - Sets a target price for a user's wishlist item.

8.4 Review APIs

- `GET /api/v1/products/{id}/reviews/sentiment` - Returns the aggregated Quality Score and the NLP distribution (e.g., 70% positive, 30% negative).

9. 🧭 User Flow / Use Cases

9.1 Search Product Flow (Bilingual)

1. User enters a query in Roman Urdu (e.g., "sasta laptop").
2. The Node.js backend passes the query to the NLP transliteration service.

3. The service converts the intent to English ("cheap laptop") or searches the native Urdu script.
4. The backend queries MongoDB. If no recent data exists, Puppeteer navigates to target store search engines, enters the translated keywords, and extracts the resulting DOM elements.
5. React renders the aggregated results.

9.2 Compare Prices Flow

1. User clicks a search result, entering the Product Details Page.
2. The app fetches the `price_sources` array from MongoDB.
3. Chart.js renders a visual graph comparing the historical and current prices of the item across Daraz, PriceOye, and Telemart.

9.3 Add Review Flow & Sentiment Analysis

1. User reads existing reviews scraped from the web.
2. The UI displays the NLP-calculated Quality Score, showing whether past buyers' sentiments were generally positive or negative based on TF-IDF and classification algorithms.

10. 🖼️ UI/UX Design

10.1 Wireframes

- **Search Interface:** A clean, dual-language supporting search bar prominently placed on the home screen.
- **Product Details Page (PDP):** A split-view layout. The left column displays the product image and dynamic specifications. The right column highlights the "Best Deal" price comparison chart and the NLP Quality Score gauge.

10.2 Key Screens

1. **Landing Page:** Search bar, trending daily drops, and category quick-links.
2. **Search Results:** Grid view of products with an at-a-glance price range.
3. **Wishlist Dashboard:** A private user area tracking saved items and triggering alerts when a scraped price falls below the user's threshold.

11. 🧪 Testing Strategy

11.1 Unit Testing

- Test the NLP transliteration logic to ensure Roman Urdu strings are correctly converted and accurately reflect consumer intent.
- Test the Product Matching algorithm's scoring accuracy using mock JSON data.

11.2 Integration Testing

- Verify the data pipeline: Ensure Kaggle CSV datasets are correctly parsed, transformed, and seeded into MongoDB collections.
- Validate the Puppeteer extraction process to ensure it can successfully bypass cookie-consent popups and extract data from a target site's search bar.

11.3 User Testing

- Evaluate the ReactJS UI responsiveness during heavy background scraping tasks, ensuring the user is given proper loading states rather than experiencing browser freezing.

12. 🚀 Deployment & Maintenance

12.1 Deployment Plan

- **Backend/Database:** Deploy MongoDB via MongoDB Atlas for easy cloud management. Host the Node.js/Express API on platforms like Heroku, Render, or AWS Elastic Beanstalk.
- **Frontend:** Deploy the ReactJS application to Vercel or Netlify for fast edge-network delivery.

12.2 Monitoring

- Implement robust error handling for Puppeteer workers. If a target e-commerce site updates its DOM structure, the Node.js worker must log the failure so developers can update the extraction logic.

12.3 Future Enhancements

- Integrate vision-based AI scraping tools to autonomously find prices and "Add to Cart" buttons without relying on rigid CSS selectors, significantly reducing future maintenance overhead.